

TRABALHO SEMESTRAL – ORIENTAÇÃO A OBJETOS	
Bacharelado em Ciência da Computação	Período: 3º
Professor: Rodrigo Martins Pagliares	Apresentação: 29 de junho de 2026 (Lab-B207)

`Data da última atualização: 09/06/2026

Instruções

Este trabalho poderá ser realizado em grupos de até 5 (cinco) estudantes.

Avaliação

A avaliação será realizada presencialmente no laboratório B207 e será composta pelos seguintes critérios:

- Código-fonte: peso 0,3;
- Apresentação do trabalho: peso 0,3;
- Capacidade de responder às perguntas formuladas pelo professor: peso 0,4.

O estudante que não comparecer à apresentação poderá obter, no máximo, a nota correspondente à avaliação do código-fonte.

Objetivos do Trabalho

O trabalho do semestre possui dois objetivos principais:

1. Complementar os conhecimentos de orientação a objetos adquiridos pelos alunos, anteriormente concentrados na implementação de requisitos funcionais, por meio da implementação de funcionalidades técnicas e requisitos de qualidade, como interface gráfica com o usuário, segurança, distribuição, persistência de dados e logging. Esses elementos serão organizados e desenvolvidos por meio de histórias de usuário e histórias técnicas.
2. Incentivar a adoção dos princípios da Engenharia de Software Aprimorada por IA (AI-Augmented Software Engineering), avaliando os ganhos de produtividade proporcionados pelo uso de Modelos de Linguagem de Grande Escala (LLMs) no apoio às atividades de engenharia de software. Ao mesmo tempo, busca-se reforçar a importância da participação ativa do ser humano no processo de desenvolvimento, validação e tomada de decisões, evitando a exclusão do ser humano do ciclo de desenvolvimento ("human-in-the-loop").

O estudante que realizar o trabalho integralmente terá a oportunidade de aplicar, na prática, diversos conceitos abordados ao longo da disciplina, incluindo análise de requisitos por meio de histórias de usuário, design orientado a objetos utilizando Diagramas de Sequência UML, aplicação dos princípios GRASP e utilização de princípios fundamentais de design de software.

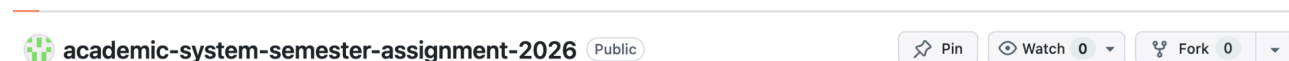
Uso de Git e GitHub

O uso de Git e GitHub não é obrigatório nesta disciplina, uma vez que essas tecnologias serão estudadas com maior profundidade na disciplina Gestão do Ciclo de Vida da Aplicação.

Entretanto, grupos que desejarem utilizar Git e GitHub para facilitar o trabalho colaborativo e reduzir problemas de integração de código são encorajados a fazê-lo. Nesse caso, os estudantes estão à vontade para utilizar LLMs como ferramenta de apoio à aprendizagem. O domínio dos conceitos básicos de Git e GitHub normalmente requer apenas algumas horas de estudo e pode trazer benefícios significativos para o desenvolvimento em equipe.

<https://github.com/pagliares/academic-system-semester-assignment-2026.git>

Dica: Caso opte por usar o git, crie uma conta no GitHub e faça o fork deste repositório. O processo de fork faz uma cópia do repositório para sua conta pessoal no GitHub (veja opção à direita no print abaixo).



Observação sobre o Tamanho das Histórias de Usuário e Histórias Técnicas

De modo geral, procurei definir histórias de usuário e histórias técnicas de pequeno porte, que, em um cenário tradicional de desenvolvimento sem o auxílio de IA, poderiam ser implementadas em poucas horas de trabalho (tipicamente entre uma e duas horas).

Consequentemente, muitas dessas histórias podem ser implementadas por meio de um número reduzido de alterações no código-fonte, frequentemente correspondendo a poucos *commits* para os grupos que optarem por utilizar Git como sistema de controle de versão. Com o apoio de ferramentas baseadas em IA, o tempo necessário para implementação pode ser significativamente reduzido.

É importante destacar que, em projetos reais de desenvolvimento de software, várias das histórias apresentadas neste trabalho provavelmente seriam tratadas como tarefas ou subtarefas de uma história de usuário maior. Em métodos ágeis como Scrum, por exemplo, uma história de usuário priorizada pelo Product Owner é frequentemente decomposta pelo time de desenvolvimento em tarefas menores para facilitar o planejamento, a implementação e o acompanhamento do progresso.

A adoção de histórias menores neste trabalho foi uma decisão intencional, com o objetivo de facilitar o gerenciamento das atividades, permitir um acompanhamento mais granular da evolução do projeto e proporcionar aos estudantes uma maior exposição a diferentes funcionalidades técnicas e requisitos de qualidade ao longo do semestre.

Exercícios

Exercício 1:. Faça uma “leitura por alto” do conteúdo do repositório com as histórias de usuário a serem implementadas no semestre.

<https://github.com/pagliares/academic-system-semester-assignment-2026>

Exercício 2. Implemente, com auxílio de LLMs/Chatbots/IDEs/Assistentes de IA, as 61 histórias de usuário presentes no repositório.

Dica: Eu implementei todas as histórias de usuário e esta foi a sequência usada por mim.

#	Code	User story title
01	US-2361	Cadastrar avaliações em turmas
02	TUS-2362	Persist class assessments to TXT File
03	US-2363	Register classes through keyboard input
04	US-2364	Manage academic system through command line menu
05	US-2365	Refactor domain model using Lombok
06	US-2366	Authenticate users and authorize actions based on roles
07	US-2367	Handle academic domain errors with custom exceptions
08	US-2368	Handle keyboard input errors with custom exceptions
09	US-2369	Handle authentication and authorization errors with custom exceptions
10	TUS-2370	Refactor menu operations into AcademicSystemController
11	TUS-2371	Validate academic domain objects using Jakarta Bean Validation
12	US-2372	Configure persistence type as administrator
13	US-2373	Save academic data to XML file
14	US-2374	Save academic data to JSON file
15	US-2375	Generate class assessment summary report
16	US-2376	Generate assessment weight report
17	US-2377	Generate persistence configuration report
18	US-2378	Role-based dynamic menu rendering
19	US-2379	Logout
20	US-2380	Display role-specific sequential menus

21	US-2381	Deliver academic system with Docker
22	TUS-2382	Define equality for identifiable domain objects
23	TUS-2383	Configure automated testing infrastructure (Automated unit tests)
24	TUS-2384	Test identifiable domain object equality (Automated unit tests)
25	TUS-2385	Test academic domain validation
26	S-2386	Test authentication behavior
27	US-2387	Test authorization behavior
28	US-2388	Test report generation
29	US-2389	Test persistence repositories
30	TUS-2390	Configure application logging infrastructure
31	TUS-2391	Log authentication and logout events
32	TUS-2392	Log authorization failures
33	TUS-2393	Log persistence operations
34	TUS-2394	Log report generation
35	TUS-2395	Verify logging infrastructure behavior (Automated unit tests)
36	TUS-2396	Introduce ClassService
37	TUS-2397	Introduce AssessmentService
38	TUS-2398	Introduce PersistenceService
39	TUS-2399	Introduce ReportService
40	TUS-2400	Simplify AcademicSystemController
41	TUS-2401	Test ClassService behavior (Automated unit tests)
42	TUS-2402	Test AssessmentService behavior (Automated unit tests)
43	TUS-2403	Test PersistenceService behavior (Automated unit tests)
44	TUS-2404	Test ReportService behavior (Automated unit tests)
45	TUS-2405	Test AcademicSystemController delegation behavior (Automated unit tests)
46	TUS-2406	Configure JavaFX application infrastructure
47	TUS-2407	Create JavaFX login screen
48	TUS-2414	Introduce AuthenticationController for JavaFX login (Refactoring)
49	TUS-2409	Create JavaFX class registration screen
50	TUS-2408	Create JavaFX role-based main screen

51	US-2410	Create JavaFX assessment registration screen
52	TUS-2411	Create JavaFX report screen
53	TUS-2412	Create JavaFX persistence configuration screen
54	TUS-2413	Create JavaFX class and assessment visualization screen
55	TUS-2415	Configure CI pipeline with GitHub Actions
56	TUS-2416	Generate test coverage reports
57	TUS-2417	Publish Docker image automatically
58	TUS-2418	Configure pull request validation workflow
59	TUS-2419	Configure release workflow

Exercício 3: Refatorar a aplicação desenvolvida no exercício 2, usando LLMs, migrando a solução para uma segunda linguagem de programação orientada a objetos à escolha do(s) estudante(s).

Bom trabalho a todos(as)!